

Learning Grasp Targets for Dense Log-Pile Clearing with a Grasp-and-Remove Crane Simulator

George Sideris*, Inna Sharf*

*Department of Mechanical Engineering, McGill University, Montreal, Canada

george.sideris@mail.mcgill.ca

Abstract—Clearing dense log piles is a contact-rich manipulation problem with a large, evolving state space and a difficult exploration landscape. In this paper, we study grasp target selection for a forestry log loader in simulation, focusing on the decision of where and how to orient a grapple to clear piles of up to 200 rigid-body logs in frictional contact. We introduce a *grasp-and-remove* formulation in which each environment step corresponds to a complete pick attempt: a policy selects a grasp target (x, y, z, ψ) in the crane base frame, a fixed finite-state controller executes the approach and lift, and grasped logs are despawned. This design isolates high-level targeting from low-level execution. Policies observe depth-derived point clouds of the segmented log pile, processed by a PointNet encoder, removing the need for privileged per-object state. We compare reinforcement learning (RL) from scratch using PPO, a strong heuristic baseline, behavior cloning (BC) from heuristic demonstrations, and BC-initialized RL fine-tuning (BC+RL). From-scratch RL maximizes per-grasp reward but converges to local optima that fail in the endgame, when remaining logs are sparse and targeting errors compound; BC initialization provides a policy that covers the full clearing trajectory, enabling RL fine-tuning to improve per-grasp performance while preserving the expert’s pile-clearing strategy. **TODO: Update with final comparative results once fixed-seed evaluation is complete.** We analyze failure modes and identify the endgame distribution shift—where remaining logs form a sparse, irregular layer—as a primary challenge for from-scratch RL in this setting.

Index Terms—robot learning, multi-object grasping from dense clutter, behavior cloning, reinforcement learning, contact-rich simulation

I. INTRODUCTION

Forestry operations increasingly seek autonomy for heavy machinery to improve safety and productivity [?]. A key subtask is log-pile clearing: repeatedly grasping multiple logs from a rack or pile and removing them. This setting is challenging for learning-based methods because (i) the scene is densely cluttered with up to 200 rigid-body logs in frictional contact, (ii) each grasp changes the pile geometry, making future grasps harder to predict, and (iii) decisions must generalize across diverse pile configurations.

This paper focuses on a single, central question: *where should the crane place and orient the grapple to clear the pile efficiently?* We deliberately separate this high-level targeting decision from low-level motion execution. Concretely, we implement a grasp-and-remove simulator in Isaac Lab [?] where each environment step is one complete pick attempt. A learned policy outputs a grasp target (x, y, z, ψ) in the crane base frame, and a fixed finite-state machine (FSM)

executes the approach, close, and lift. After lifting, logs within a proximity threshold of the grapple are removed, allowing us to measure clearing progress without modeling deposition dynamics.

This factorization has two benefits. First, it reduces the horizon of the learning problem: the policy is rewarded once per grasp attempt, rather than for every low-level control step. Second, it makes comparisons between methods clean: all methods share the same low-level controller and differ only in target selection.

A central finding is that from-scratch RL converges to local optima of the per-grasp reward: it learns to pick easily accessible logs but fails in the *endgame*, when the pile is sparse and small targeting errors lead to empty grasps. BC initialization provides a policy that covers the full clearing trajectory—including difficult endgame configurations—enabling RL fine-tuning to improve per-grasp performance while preserving pile-clearing ability.

Contributions.

- A grasp-and-remove crane environment in Isaac Lab for contact-rich log-pile clearing at scale (up to 200 logs), with a fixed FSM controller that isolates the targeting decision.
- A depth-based point cloud observation pipeline—segmented, back-projected, and encoded by a PointNet [?]¹—that removes the need for privileged per-object state, with a privileged-pose ablation to quantify the perception gap.
- An empirical comparison of from-scratch RL, heuristic targeting, BC, and BC+RL fine-tuning, showing that from-scratch RL converges to local optima that fail in the endgame, while BC initialization enables RL to optimize per-grasp performance without sacrificing pile-clearing ability.

II. RELATED WORK

Grasping in clutter and pile manipulation. Learning-based grasp detection has progressed from single isolated objects to cluttered, multi-object scenes. Deep grasp prediction networks such as GG-CNN [?] and the multi-grasp architecture of Kumra and Kanan [?] predict grasp pose directly from visual input, using cosine-sine angle encodings to handle grasp symmetry. Pile manipulation and sequential decluttering have been studied in bin picking [?], [?], but typically involve rigid objects of moderate count (<50). Our setting differs in

scale (up to 200 elongated logs) and in the multi-object nature of each grasp (5–20 logs per attempt). For 3D perception, PointNet [?] processes unordered point sets via per-point MLPs and symmetric pooling; we adopt this architecture to encode depth-derived point clouds into a latent representation for grasp target prediction.

RL for contact-rich manipulation. Reinforcement learning has been applied to contact-rich tasks including dexterous in-hand manipulation [?] and assembly [?]. These works demonstrate that RL can discover complex contact strategies in simulation, but typically involve single objects or small object counts. RL for heavy machinery is comparatively underexplored; recent work has applied RL to excavator control [?] and forestry crane motion planning [?], but these focus on trajectory optimization rather than grasp target selection in dense clutter.

Imitation learning and BC+RL. Behavior cloning [?] provides a simple supervised approach to policy learning from demonstrations, but suffers from distribution shift as the policy encounters states not seen during training [?]. DAgger [?] addresses this by iteratively collecting on-policy data with expert labels. An alternative is to use BC to initialize an RL policy and fine-tune with on-policy experience [?], [?]. In our setting, BC+RL is particularly well suited because the expert is a heuristic (not a human), so the bottleneck is not label collection but rather improving *beyond* the expert—which RL can achieve but DAgger cannot.

Simulation platforms. We build on Isaac Lab [?], which provides GPU-accelerated physics simulation via PhysX and supports large-scale parallel environments. For RL training we use RSL-RL [?] with PPO [?].

III. TASK FORMULATION

A. Grasp-and-Remove MDP

We formulate the task as an episodic Markov decision process $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ where each *environment step* corresponds to a complete grasp attempt rather than a low-level control tick. The state $s_t \in \mathcal{S}$ is the current pile configuration (observed via point cloud or privileged poses; Section ??). The action $a_t \in \mathcal{A} \subset \mathbb{R}^5$ specifies a grasp target—position (x, y, z) and grapple yaw ψ —in the crane base frame (Section ??). The transition $T(s_{t+1}|s_t, a_t)$ is determined by the physics simulation: a fixed finite-state machine (FSM) executes the approach, close, and lift, and grasped logs are removed from the scene. The scalar reward $r_t = R(s_t, a_t)$ is issued once per grasp attempt based on the measured outcome (Section ??). We use a discount factor $\gamma = 0.99$. An episode terminates when all logs have been removed (grasped or knocked off the rack) or after a fixed horizon of $H = 30$ grasp attempts. The horizon is chosen to give the policy comfortable headroom beyond the minimum needed: the grapple can hold up to 20 logs per grasp, and a strong heuristic baseline clears 200-log piles in **TODO: 15–20** attempts, so $H = 30$ allows roughly 50–100% additional attempts while keeping episodes tractable.

At the start of each step the crane is in a standardized HOVER configuration above the rack. This formulation in-

tentionally *does not* include deposition; removed logs are teleported out of the scene. Our aim is to study targeting and grasping metrics without conflating them with downstream placement complexity.

B. Observations

The policy observes a segmented point cloud of the log pile, produced by a four-stage pipeline:

- 1) A depth camera mounted on the crane renders the scene from above the rack.
- 2) A semantic segmentation mask isolates log pixels from the depth image. In simulation this mask comes from the renderer’s semantic channel; on a real crane it would be provided by a learned perception module—recent work has demonstrated instance-level log segmentation from RGB in forestry settings [?].
- 3) The masked depth image is back-projected to 3D and transformed into the crane base frame.
- 4) Farthest Point Sampling (FPS) downsamples the cloud to a fixed $N_p = 1024$ points, yielding a (1024×3) observation tensor.

Privileged pose baseline. As an ablation, we also evaluate policies that observe privileged per-object state: the top- K log poses sorted by height (highest first), each represented as (x, y, z, ψ) in the crane base frame. Removed logs are excluded and remaining slots are padded with -1 . With $K = 32$ this yields a compact 128D vector. This baseline isolates the effect of the perception gap: any performance difference between pose and point cloud policies is attributable to the observation representation rather than the learning algorithm.

C. Actions and Workspace Scaling

The policy outputs an unconstrained 5D vector $\tilde{a}_t \in \mathbb{R}^5$. The first three components encode position and the remaining two encode grapple yaw via a cosine–sine pair.

Position. Each position component is squashed via tanh and then linearly mapped to the per-environment workspace bounds:

$$p_i = \frac{(\tanh \tilde{a}_i + 1)}{2} (b_i^{\max} - b_i^{\min}) + b_i^{\min}, \quad i \in \{x, y, z\}, \quad (1)$$

where $b_i^{\min}, b_i^{\max}$ are the rack-aligned workspace limits. These bounds are computed from the rack geometry and serve a dual purpose. First, they define the policy’s reachable target space, ensuring that all output actions map to physically executable grasp poses. Second, logs that drift outside these bounds during a grasp attempt (e.g., knocked off the rack by grapple contact) are automatically removed before the next targeting decision and counted separately (see Section ??), preventing unreachable logs from accumulating. The heuristic expert (Section ??), which bypasses the policy’s action scaling and targets log poses directly, operates within the same spatial bounds.

Yaw (cosine–sine encoding). A cylindrical log at yaw ψ is physically identical to the same log at $\psi + \pi$. A scalar yaw representation forces the network to learn this equivalence

from data; instead, following the grasp-angle encoding of Morrison et al. [?], we represent yaw as

$$(\cos 2\psi, \sin 2\psi), \quad (2)$$

which has period π and thus maps symmetric orientations to the same point. The policy outputs two unconstrained scalars $(\tilde{a}_4, \tilde{a}_5)$; after squashing, the yaw is recovered as

$$\psi = \frac{1}{2} \operatorname{atan2}(\tanh \tilde{a}_5, \tanh \tilde{a}_4). \quad (3)$$

In our imitation pipeline, expert targets are encoded in the same space: positions are normalized and passed through $\operatorname{arctanh}$, and yaw is stored directly as $(\cos 2\psi, \sin 2\psi)$. This ensures BC and RL policies operate in an identical action space.

IV. ENVIRONMENT AND CONTROLLER

A. Low-level Execution FSM

All methods share the same FSM controller, which runs at the physics timestep inside each step. The FSM implements a pick sequence with phase timeouts and dwell-based convergence checks: HOVER $\rightarrow$ ALIGN $\rightarrow$ DESCEND $\rightarrow$ CLOSE $\rightarrow$ LIFT $\rightarrow$ OPEN $\rightarrow$ return to HOVER. In all experiments in this paper, the arm is controlled using Isaac Lab’s task-space *Differential IK* controller [?] (damped least-squares, position command type), which computes joint position targets from the current end-effector pose and Jacobian. The grapple jaws are controlled using a simple PD law that outputs joint *velocity* targets with saturation.

B. Outcome Evaluation and Log Removal

After lifting (LIFT), we estimate grasp outcome by measuring proximity and orientation of logs relative to the basegrapple body:

- **Logs grasped (g):** number of active logs whose centers lie within a proximity radius of the grapple (default 1.5 m).
- **Alignment (a):** for each grasped log, we compute the absolute dot product between the grapple’s local Y axis and the log’s length axis (also modeled along local Y), and apply a sharp nonlinearity $a_i = |\langle \hat{y}_g, \hat{y}_\ell \rangle|^8$. We report the mean $a = \frac{1}{g} \sum_i a_i \in [0, 1]$.
- **Stability (s):** we compute the grapple “levelness” as the Z component of the grapple’s up vector in world coordinates (dot with world up), clamped to $[0, 1]$ and sharpened as $s = (\max(0, \hat{z}_g^\top \hat{z}_w))^4$.

Logs within the grasp radius are then removed by teleportation and marked as inactive.

C. Reward

Reward is issued once per grasp attempt after LIFT based on the measured outcome (g, a, s) . A failed grasp ($g = 0$) receives a fixed penalty $r = \rho_{\text{fail}}$ (default -1).

For successful grasps, we define an *efficiency* term e as either raw throughput ($e = g$) or as a normalized fraction

$$e = \min\left(\frac{g}{\min(g_{\max}, \max(1, n_{\text{avail}}))}, 1\right), \quad (4)$$

where n_{avail} estimates the number of logs available in a rack slice at the target y , and $g_{\max} = 20$ caps the physically graspable logs.

We evaluate two per-grasp reward families. In the **additive** variant:

$$r_{\text{add}} = e + a + s, \quad (5)$$

and in the **multiplicative** variant (MR, our primary configuration):

$$r_{\text{mult}} = g \cdot a \cdot s, \quad (6)$$

where g is raw throughput (number of logs grasped). With the normalized variant (MN, `normalize_reward=True`), efficiency e replaces g and is scaled by a factor of 10; this variant is not the focus of the main experiments.

Note that the reward is entirely per-grasp; there is no episode-level completion bonus. Global pile-clearing ability thus emerges from the policy’s targeting strategy rather than from explicit incentives to clear.

Knocked-off log tracking. Logs that move outside the workspace bounds during a grasp attempt (e.g., knocked off the rack by grapple contact) are removed from the scene before the next targeting decision and counted separately. These logs are *not* credited toward the grasp reward g for that attempt, but they do reduce the remaining pile count. We report knocked-off percentage as an auxiliary metric (Section ??).

D. Simulation Notes

We implement the environment in Isaac Lab and run PhysX GPU dynamics for scalable data collection. We use conservative contact modeling—simplified collision geometry on the grapple, tuned contact and rest offsets, and damping limits for logs—to reduce interpenetration and numerical jitter while maintaining high simulation throughput.

V. LEARNING AND BASELINES

A. Heuristic Expert

We implement a strong heuristic expert that targets a high, graspable log. At HOVER, the expert selects the highest available log (with an option to sample among the top- k), targets its center (x, y, z) , and computes a yaw that aligns the grapple with the log’s principal axis, choosing the 180° flip that minimizes rotation. During BC data collection, we optionally add independent Gaussian noise to each position component (σ_{pos} in cm) and to the yaw (σ_ψ in degrees) for regularization.

B. Behavior Cloning

We collect demonstration pairs (o_t, a_t) from successful expert grasps only (failed grasps are discarded), where o_t is the point cloud observation and a_t is the expert action in the unconstrained space (positions via $\operatorname{arctanh}$, yaw via $\cos 2\psi, \sin 2\psi$ as described in Section ??).

The policy uses a PointNet encoder [?] consisting of per-point MLPs ($3 \rightarrow 64 \rightarrow 128 \rightarrow 256$) with batch normalization and ELU activations, followed by max-pooling over points and a fully connected layer to a 256D latent vector. An actor MLP

TABLE I: Network architectures for point cloud (PCD) and privileged pose observation types. All hidden layers use ELU activations. The PointNet encoder uses batch normalization.

Component	BC/RL (PCD)	RL (Pose)
Encoder	PointNet: $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$ (BN, ELU), MaxPool	MLP: $128 \rightarrow 256 \rightarrow 128 \rightarrow 64$
Actor	MLP: $256 \rightarrow 128 \rightarrow 64 \rightarrow 5$	Shared encoder $\rightarrow 64 \rightarrow 5$
Critic	Independent PointNet encoder + MLP head	MLP: $256 \rightarrow 128 \rightarrow 64 \rightarrow 1$
Output	5D: $(x, y, z, \cos 2\psi, \sin 2\psi)$	

($256 \rightarrow 128 \rightarrow 64 \rightarrow 5$) maps the latent to the action. For the privileged-pose ablation, we replace the encoder with a three-layer MLP ($128 \rightarrow 256 \rightarrow 128 \rightarrow 64$) with ELU activations [?].

Table ?? summarizes the network architecture for both observation types.

All BC models are trained with mean-squared error using AdamW [?] (learning rate 10^{-4} , weight decay 10^{-4}) and ReduceLROnPlateau scheduling (factor 0.5, patience 10 epochs), with an 85/15 train/validation split and early stopping (patience 30 epochs). **TODO: State final dataset size (number of episodes and successful-grasp samples).**

C. Reinforcement Learning (RL)

We train policies with Proximal Policy Optimization (PPO) [?] using the RSL-RL library [?]. PPO optimizes a stochastic Gaussian policy $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma^2))$, where the actor network μ_{θ} outputs the mean action and $\sigma \in \mathbb{R}^{|A|}$ is a learnable, state-independent, per-action-dimension standard deviation parameter that controls exploration noise. At initialization, all components of σ are set to σ_0 ; during training, σ is updated by gradient descent alongside the actor weights, typically decreasing as the policy converges. A separate critic network $V_{\phi}(s)$ estimates the state-value function for advantage computation via Generalized Advantage Estimation (GAE) [?].

Hereafter, we refer to from-scratch training as **RL** and BC-initialized fine-tuning as **BC+RL**. The actor uses the same PointNet encoder and actor MLP as the BC policy; the critic uses an independent PointNet encoder with its own MLP head. For the privileged-pose ablation, both actor and critic are separate MLPs with hidden dimensions [256, 128, 64] and ELU activations. Key hyperparameters: $\sigma_0 = 1.0$, clipping ratio $\epsilon = 0.2$, entropy coefficient 0.01, learning rate 3×10^{-4} with adaptive KL-based scheduling (target KL 0.01), GAE with $\lambda = 0.95$, 4 steps per environment per iteration, 5 learning epochs with 4 mini-batches per epoch, and gradient clipping at norm 1.0.

D. BC+RL Fine-Tuning

For BC+RL, we initialize the actor weights from a trained BC checkpoint, initialize the critic randomly, and reset the optimizer state. The initial exploration noise is set to $\sigma_0 =$

TODO: Insert environment visualization: crane + rack + 200 logs, showing the grapple above the pile in HOVER configuration.

Fig. 1: The grasp-and-remove crane environment. A log loader crane is positioned above a rack containing up to 200 cylindrical logs. At each step, the policy selects a grasp target and a fixed FSM controller executes the pick sequence.

TODO: Insert point cloud pipeline figure: (a) depth image, (b) segmentation mask, (c) back-projected 3D points, (d) FPS-downsampled 1024-point cloud.

Fig. 2: Observation pipeline. A crane-mounted depth camera renders the scene (a), a semantic mask isolates log pixels (b), the masked depth is back-projected to 3D (c), and farthest point sampling produces the fixed-size (1024×3) input (d).

0.3 (reduced from the RL default of 1.0): because the BC actor already outputs near-expert actions, a large σ_0 would add destructive noise and erase the pretrained behavior, while a small σ_0 keeps sampled actions close to the BC mean and allows PPO to refine rather than overwrite the initialization.

The key insight is that from-scratch RL, optimizing per-grasp reward, converges to local optima that pick easily accessible logs but fail in the endgame. BC initialization places the policy in a region of parameter space that covers the full clearing trajectory, and the reduced σ_0 ensures that early RL updates preserve this strategy.

VI. EXPERIMENTS

A. Setup

All experiments use the same crane model, rack geometry, and FSM controller. Piles contain 200 cylindrical logs (~ 0.3 m diameter, ~ 3 m length) spawned in layered grid patterns. The log count is fixed across all experiments so that clearing metrics are directly comparable between methods; domain randomization varies only the pile geometry (spawn pattern, row jitter, and layer offsets).

Each episode runs for a fixed budget of $H = 30$ grasp attempts (see Section III-A for the horizon rationale). An episode terminates early if all logs have been removed—either grasped and lifted or knocked outside the workspace bounds. For evaluation, we run **TODO: N** episodes across **TODO: E** parallel environments on identical random seeds to ensure each method faces the same pile configurations. All metrics are reported as per-episode mean $\pm$ standard deviation.

B. Metrics

We report per-grasp and per-episode metrics, all as per-episode mean $\pm$ standard deviation unless noted:

- **Grasp success rate:** fraction of grasps with $g > 0$.
- **Throughput:** mean logs grasped per successful grasp.
- **Alignment:** grasp-count-weighted mean $a \in [0, 1]$ per episode.

TODO: Insert clearing progression curves: fraction of pile cleared (mean $\pm$ shaded std across episodes) vs. grasp attempt number, one curve per method (Heuristic, RL, BC, BC+RL). This is the primary results figure.

Fig. 3: Clearing progression. Each curve shows the fraction of the 200-log pile cleared as a function of grasp attempt number (mean $\pm$ std over **TODO:** N episodes). **TODO:** Describe key observations: where methods diverge, RL plateau, BC+RL endgame.

- **Stability:** grasp-count-weighted mean $s \in [0,1]$ per episode.
- **Pile clearing rate:** fraction of initial logs removed by episode end (grasped or knocked off).
- **Full-clear rate:** fraction of episodes where all logs are removed.
- **Knocked-off percentage:** fraction of initial logs knocked outside workspace bounds.
- **Grasps to 50%/90% cleared:** number of grasp attempts to reach 50% and 90% pile clearing, computed over episodes that reach each threshold. These efficiency metrics capture pile-level performance without requiring full clears (which are rare for some methods).
- **Episode reward:** cumulative per-grasp reward over the episode.

Our primary visualization is the *clearing progression curve*: for each method, we plot fraction of pile cleared (mean $\pm$ std across episodes) as a function of grasp attempt number (Fig. ??). This curve captures the full clearing dynamics—where methods diverge, where RL plateaus, and how BC+RL maintains clearing momentum into the endgame—and is strictly more informative than discrete threshold metrics alone. Grasps-to-50% and grasps-to-90% serve as compact summary statistics derived from these curves.

VII. RESULTS

A. Overall Performance

TODO: Fill in Table ?? with fixed-seed evaluation results for all methods.

Qualitatively, we observe three regimes: (i) **early clearing**, where dense piles yield high-throughput grasps for all methods; (ii) **mid clearing**, where policies diverge in how they target remaining dense regions; and (iii) **endgame clearing**, where remaining logs form a thin sparse layer and targeting becomes sensitive to small errors.

TODO: Describe results once evaluation is complete. Key comparisons: (1) Does BC+RL achieve faster clearing progression (lower grasps-to-50%/90%) than the heuristic? (2) How does from-scratch RL perform on clearing rate and full-clear rate? (3) Do the clearing progression curves (Fig. ??) show the expected divergence in the endgame?

B. Local Optima and Endgame Failure

From-scratch RL achieves high per-grasp reward by targeting easily accessible logs in the dense early phase, but this

TODO: Insert training curves: (a) episode reward vs. training iterations for RL and BC+RL, (b) clearing percentage vs. training iterations for RL and BC+RL. Optionally (c) σ decay.

Fig. 4: Training dynamics. (a) Episode reward: RL plateaus at a local optimum; BC+RL starts at BC performance and improves. (b) Clearing percentage: RL never reaches high clearing rates; BC+RL maintains the expert’s clearing ability.

TODO: Insert per-remaining-logs analysis: mean logs grasped per attempt (y-axis) vs. number of remaining logs (x-axis), one curve per method. Shows endgame degradation.

Fig. 5: Per-grasp throughput as a function of remaining logs. As the pile thins, from-scratch RL throughput degrades sharply, while BC+RL maintains effective targeting into the endgame.

strategy fails in the endgame when the pile thins out. **TODO:** Report: full-clear rate for RL vs. BC+RL; training curves showing reward plateau. Because the per-grasp reward does not explicitly penalize endgame failure, RL has no gradient signal pushing it toward globally effective strategies. BC initialization provides a policy that already handles the full clearing trajectory, including sparse endgame configurations, giving RL a starting point from which fine-tuning improves per-grasp metrics without sacrificing pile-clearing ability.

C. Training Dynamics

We examine training dynamics through two complementary views: (i) episode reward vs. training iterations, and (ii) clearing percentage vs. training iterations. For from-scratch RL, episode reward plateaus at a local optimum where per-grasp performance is high but clearing stalls—the policy never learns to handle the endgame. For BC+RL, episode reward starts at the BC performance level and rises above it as RL fine-tuning improves per-grasp metrics, while clearing percentage is maintained or improved from the BC initialization. The clearing percentage view is particularly revealing: from-scratch RL never reaches high clearing rates regardless of training duration, while BC+RL maintains the expert’s clearing ability throughout training. **TODO:** Optionally show σ (exploration noise) decay curves to illustrate convergence.

D. Endgame Failure Analysis

To understand why from-scratch RL underperforms, we analyze per-grasp performance as a function of remaining logs.

A consistent pattern is *endgame distribution shift*: late in an episode, the pile geometry is sparse and irregular, and small targeting errors lead to empty grasps. Because the per-grasp reward does not account for the global pile state, inefficient late grasps accumulate and prevent full clearing within the episode budget. BC initialization mitigates this by biasing the policy toward expert-like targets that remain effective in sparse conditions.

TABLE II: Evaluation results across **TODO: N** fixed pile configurations. All values are per-episode mean $\pm$ std. Grasps to 50%/90% are computed over episodes reaching each threshold.

Method	Obs.	Grasp Succ. %	Through-put	Align.	Stab.	Clear Rate %	Full Clear %	Knocked Off %	Grasps to 50%	Grasps to 90%
Heuristic	Pose	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:
RL	PCD	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:
BC	PCD	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:
BC+RL	PCD	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:
RL	Pose	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:	TODO:

VIII. DISCUSSION AND LIMITATIONS

Why not DAgger? DAgger [?] addresses BC’s distribution shift by iteratively collecting on-policy data with expert labels. In our setting, the expert is a cheap heuristic (not a human), so the label-collection bottleneck that motivates DAgger does not apply. More importantly, DAgger can only match the expert, whereas BC+RL can potentially *surpass* it by optimizing the reward directly. Our BC+RL approach thus addresses both distribution shift (via on-policy experience) and performance ceiling (via reward optimization).

Curriculum learning. An alternative to BC initialization is curriculum learning over pile density—training first on small piles where clearing is easy and gradually increasing to 200 logs. However, sparse piles differ from dense piles not only in difficulty but in structure: contact interactions, log accessibility, and viable grasp strategies change qualitatively with pile size. BC+RL avoids this domain gap by initializing from demonstrations collected directly in the target distribution.

TODO: If curriculum experiment is run, report results here.

Limitations. Our grasp-and-remove design isolates targeting and enables fast experimentation, but has limitations. First, grasp success is approximated by proximity after lift, rather than explicit contact constraints. Second, removed logs are teleported rather than placed, so the task omits deposition planning and re-contact dynamics. Third, while point cloud observations remove the need for privileged state, the PointNet encoder operates on unordered point sets without explicit spatial structure; architectures that exploit local geometry (e.g., PointNet++, point transformers) may improve endgame targeting. Fourth, all experiments are in simulation; sim-to-real transfer with domain randomization is a natural next step.

Despite these simplifications, the environment captures essential challenges of multi-object grasping in dense clutter and reveals how per-grasp reward optimization alone leads to local optima that fail to clear piles.

IX. CONCLUSION

We presented a grasp-and-remove crane environment for studying grasp target selection in dense log-pile clearing at scale (up to 200 rigid-body logs in contact). By factorizing target choice from low-level execution, we compared heuristic targeting, behavior cloning, from-scratch RL, and BC+RL on a shared controller. Our analysis reveals that from-scratch RL

converges to local optima that fail in the endgame, while BC initialization enables RL fine-tuning to improve per-grasp performance without sacrificing pile-clearing ability. Point cloud observations achieve comparable clearing performance to privileged state, demonstrating a viable path to sensor-based deployment. **TODO: Update with final result summary, referencing clearing progression curves and grasps-to-50%/90% metrics.** Future work will explore sim-to-real transfer with domain randomization, richer point cloud architectures, and curriculum learning over pile complexity.